Design and Analysis of Algorithms

Mid-Semester Crash Course

B.Stat. 3rd year, Semester V — Lectures 1–10

Every topic in the lecture notes is presented in the same five-part rhythm:

Problem	what is being asked, stated precisely
Algorithm	the solution, as numbered steps you can reproduce
Correctness	why it works — the proof, not a gesture at one
Time complexity	the analysis, with the recurrence solved
Exam facts	everything else that gets asked about this topic

Trap boxes flag the specific mistakes that cost marks. Section 14 maps the 2022 mid-semester paper onto this material; §15 reproduces that paper and §16 solves it in full; §§17–18 are a fresh practice set with solutions.

Contents

1 The 1-Center Problem (Minimum Enclosing Circle)

Problem — 1-centre / MEC

Given points $p_1, \ldots, p_n \in \mathbb{R}^2$, find p minimising $\max_i d(p, p_i)$ — the point whose worst-case distance to the data is smallest. Equivalently: find the centre of the **minimum enclosing circle** (MEC) of the point set.

Why these are the same. If p is the centre of the MEC of radius r then $\max_i d(p, p_i) = r$. No point can do better: a p' with $\max_i d(p', p_i) < r$ would give a strictly smaller enclosing circle centred at p', contradicting minimality of r.

Algorithm — Brute force over candidate circles

The geometric hook: **three non-collinear points determine a unique circle**. The MEC's boundary must touch either 2 or 3 of the input points, so:

1. **3-point circles.** For each of the $\binom{n}{3}$ triples, form the unique circle through them.
2. **2-point circles.** For each of the $\binom{n}{2}$ pairs, form the circle having that pair as a diameter (covers the case where the boundary touches only two points).
3. For each of these $\binom{n}{2} + \binom{n}{3}$ candidates, test in $O(n)$ whether it encloses all n points.
4. Return the smallest candidate that encloses everything.

Correctness

Every enclosing circle can be shrunk until its boundary touches the point set. Once it cannot shrink further, the touching points *pin* it: either two points diametrically opposite, or three points not all on a semicircle. Both cases are in the candidate list, so the true MEC is among the candidates; taking the smallest enclosing candidate returns it. ■

Time complexity

$\binom{n}{3} = O(n^3)$ dominates $\binom{n}{2} = O(n^2)$, and each candidate costs $O(n)$ to test:

$$O(n) \cdot \left(\binom{n}{2} + \binom{n}{3} \right) = O(n) \cdot O(n^3) = \boxed{O(n^4)}.$$

Exam facts — comparisons for max, and min & max together

A separate, very examinable sub-topic on *comparison counting*.

- **Maximum alone:** exactly $n - 1$ comparisons (linear scan). This is optimal — each comparison eliminates at most one candidate, and $n - 1$ candidates must be eliminated.
- **Min and max, naive:** max in $n - 1$, then min of the remaining $n - 1$ in $n - 2$: total $2n - 3$.
- **Min and max, pairwise (better):** process elements *in pairs*. Compare the pair internally (1 comparison); compare only the smaller against the running min and only the larger against the running max (2 more). So 3 comparisons per pair:

$$3 \cdot \frac{n}{2} = \frac{3n}{2} \text{ comparisons } \left(\approx \tfrac{3n}{2} - 2 \text{ exactly} \right),$$

beating $2n - 3$. This is the classical tight bound.

- *Correctness (induction on pairs).* After k pairs the running min/max are correct for the $2k$ elements seen. A new pair (x, y) can only affect the global min through $\min(x, y)$

and the global max through $\max(x, y)$, so the internal comparison plus two updates extends the invariant to $2k + 2$.

- Faster MEC algorithms exist (Welzl's, expected linear time); $O(n^4)$ is only what the elementary argument gives. Say this if asked "can you do better?"

2 Recurrences and the Master Theorem

Problem — Solving divide-and-conquer recurrences

Given a recurrence for the running time of a recursive algorithm, produce a closed-form Θ or O bound.

Two standard unrollings

$f(n) = f(n-1) + dn \implies f(n) = \Theta(n^2)$, $\quad f(n) = 2f(n/2) + dn \implies f(n) = \Theta(n \log n)$.

The first is an arithmetic sum $d(n + (n-1) + \cdots)$; the second is exactly merge sort.

Exam facts — The Master Theorem

Applies *only* to the single-branching shape

$$T(n) = a\,T\left(\frac{n}{b}\right) + f(n), \qquad a \geq 1,\ b > 1,$$

where all a subproblems have the **same** size n/b. Compare $f(n)$ with $n^{\log_b a}$:

- **Case 1.** $f(n) = O(n^{\log_b a - \epsilon})$ for some $\epsilon > 0 \implies T(n) = \Theta(n^{\log_b a})$. (leaves dominate)
- **Case 2.** $f(n) = \Theta(n^{\log_b a}) \implies T(n) = \Theta(n^{\log_b a} \log n)$. (balanced)
- **Case 3.** $f(n) = \Omega(n^{\log_b a + \epsilon})$ for some $\epsilon > 0$ *and* the regularity condition $af(n/b) \leq cf(n)$ holds for some $c < 1$ and large $n \implies T(n) = \Theta(f(n))$. (root dominates)

Extended Case 2: if $f(n) = \Theta(n^{\log_b a} \log^k n)$ with $k \geq 0$, then $T(n) = \Theta(n^{\log_b a} \log^{k+1} n)$.

Trap — when the Master Theorem does *not* apply

It fails the moment the subproblems have **different sizes**, e.g.

$$T(n) = T(n/5) + T(7n/10) + O(n).$$

This is exactly the median-of-medians recurrence. Writing "by the Master Theorem" here is a guaranteed mark loss. Use the lemma below instead.

Exam facts — Substitution lemma for multi-branch shrinking recurrences

Claim. If $T(n) \leq T(a_1 n) + T(a_2 n) + \cdots + T(a_k n) + cn$ with each $a_i \in (0, 1)$ and

$$\alpha := a_1 + a_2 + \cdots + a_k < 1,$$

then $T(n) = O(n)$.

Proof. Guess $T(n) \leq dn$ and substitute:

$$T(n) \leq d(a_1 n) + \cdots + d(a_k n) + cn = dn\alpha + cn.$$

Choosing $d = \dfrac{c}{1-\alpha}$ gives $dn\alpha + cn = dn$, closing the induction. ■

The whole content is $\alpha < 1$: the subproblem sizes must sum to strictly less than the original. If $\alpha = 1$ you get $\Theta(n \log n)$; if $\alpha > 1$ it is superlinear.

Proving a Master-Theorem case by unrolling (exam-standard)

Problem — $T(n) = a\,T(n/b) + c\,n^k$, $T(1) = d$, with $a > b^k$

Show $T(n) = \Theta(n^{\log_b a})$. (This exact question was asked in 2022.)

Proof. Unroll for $n = b^L$, $L = \log_b n$:

$$T(n) = \sum_{i=0}^{L-1} a^i\, c \left(\frac{n}{b^i}\right)^k + a^L\, d = c\,n^k \sum_{i=0}^{L-1} \left(\frac{a}{b^k}\right)^i + d\,n^{\log_b a},$$

using $a^L = a^{\log_b n} = n^{\log_b a}$. Put $r = a/b^k$. The hypothesis $a > b^k$ gives $r > 1$, so the geometric sum is dominated by its *last* term:

$$\sum_{i=0}^{L-1} r^i = \frac{r^L - 1}{r - 1} = \Theta(r^L), \qquad r^L = \frac{a^{\log_b n}}{b^{k \log_b n}} = \frac{n^{\log_b a}}{n^k}.$$

Therefore $c\,n^k \cdot \Theta(n^{\log_b a}/n^k) = \Theta(n^{\log_b a})$, and adding $d\,n^{\log_b a}$ leaves the order unchanged:

$$\boxed{T(n) = \Theta(n^{\log_b a}).} \qquad \square$$

Exam facts — the three regimes, in one sentence each

Comparing a with b^k decides who wins:

$a > b^k$: $\Theta(n^{\log_b a})$ (leaves), $\quad a = b^k$: $\Theta(n^k \log n)$ (all levels equal), $\quad a < b^k$: $\Theta(n^k)$ (root).

The geometric series is increasing, constant, or decreasing respectively — that is the entire proof, and it is worth stating that way.

3 Linear-Time Selection (Median of Medians)

Problem — Selection

Given an unordered (multi)set $S = \{x_1, \ldots, x_n\}$ of reals and an index i, find the i-th smallest element of S. The case $i = \lceil n/2 \rceil$ is the median. We want $O(n)$ in the **worst case** — not merely in expectation, as randomised quickselect gives.

Algorithm — Median of Medians

1. **Group.** Split S into $\lceil n/5 \rceil$ groups of 5 (the last may be short).
2. **Baby medians.** Find each group's median directly in $O(1)$ (sort 5 elements). Call them $y_1, \ldots, y_k$, $k = \lceil n/5 \rceil$.
3. **Recurse for the pivot.** Recursively find the median z of $\{y_1, \ldots, y_k\}$ using this same algorithm.

4. **Partition.** Split S into $S_1 = \{x < z\}$ and $S_2 = \{x > z\}$, setting z aside.
5. **Recurse on one side.**
   - If $|S_1| = i - 1$: return z.
   - If $|S_1| \geq i$: recurse for the i-th smallest in S_1.
   - Else: recurse for the $(i - |S_1| - 1)$-th smallest in S_2.

Correctness

The split guarantee. At least $\frac{3n}{10} - 6$ elements are $\leq z$, and at least $\frac{3n}{10} - 6$ are $\geq z$.

Proof. z is the median of the $k = \lceil n/5 \rceil$ group medians, so at least $k/2$ groups have their own median $\leq z$. In each such *full* group, the median and the two elements below it are all $\leq z$ — that is 3 of the 5. Hence

$$\#\{x \leq z\} \;\geq\; 3\left(\frac{1}{2}\left\lceil\frac{n}{5}\right\rceil - 1\right) \;\geq\; \frac{3n}{10} - 6,$$

the -6 absorbing the group containing z and a possibly incomplete last group. Symmetrically for $\geq z$. ■

Consequence. $|S_1|, |S_2| \leq n - \left(\frac{3n}{10} - 6\right) = \frac{7n}{10} + 6$: neither recursive call in Step 5 ever sees more than $\frac{7n}{10} + 6$ elements.

Selection correctness. Immediate induction on $|S|$: after partitioning, the rank of z in S is exactly $|S_1| + 1$, so comparing i with $|S_1| + 1$ identifies which side contains the i-th smallest, with the rank correctly re-indexed on the right side. ■

Time complexity

Steps 1–2 cost $O(n)$. Step 3 recurses on $\lceil n/5 \rceil$; Step 5 on at most $\frac{7n}{10} + 6$:

$$f(n) \;\leq\; f\left(\frac{n}{5}\right) + f\left(\frac{7n}{10} + 6\right) + O(n).$$

This is not a Master Theorem instance — the two subproblems have different sizes. Apply the substitution lemma with $a_1 = \frac{1}{5}$, $a_2 = \frac{7}{10}$:

$$\alpha = \frac{1}{5} + \frac{7}{10} = \frac{9}{10} < 1 \quad\Longrightarrow\quad \boxed{f(n) = O(n)}.$$

Explicitly, guessing $f(n) \leq dn$:

$$f(n) \leq \frac{dn}{5} + \frac{7dn}{10} + 6d + cn = \frac{9dn}{10} + 6d + cn \leq dn$$

for d large enough relative to c (and n past a threshold, with a separate base case).

Exam facts — why groups of 5 — the classic viva question

With groups of size g, the guaranteed fraction on the small side is about $\frac{1}{2} \cdot \frac{g/2}{g}$, so the recursion total is roughly

$$\underbrace{\frac{1}{g}}_{\text{median-of-medians call}} \quad \underbrace{+\,1 - \frac{1}{2} \cdot \frac{\lceil g/2 \rceil}{g}}_{\text{larger side}}.$$

- $g = 5$: $\frac{1}{5} + \frac{7}{10} = \frac{9}{10} < 1$ ✓ the lemma applies, $O(n)$.
- $g = 3$: $\frac{1}{3} + \frac{2}{3} = 1$ *imes* not *strictly* less than 1; the recursion degrades to $\Theta(n \log n)$.
- $g = 7$ also works, but 5 is the smallest odd size that does.

Odd g keeps the group median unambiguous. **Also worth knowing:** randomised quickselect is $O(n)$ *expected* with a much smaller constant; MoM's value is the worst-case guarantee.

Exam facts — the two standard applications

- k **smallest elements in** $O(n)$ **(unordered).** Select the k-th smallest in $O(n)$, then partition in $O(n)$ and output the left side. To output them *sorted* costs $O(n + k \log k)$ — and $\Omega(k \log k)$ is forced by the sorting lower bound.
- k **numbers nearest the median in** $O(n)$**.** (i) Find the median M ($O(n)$); (ii) form $d_i = |a_i - M|$ ($O(n)$); (iii) select the k-th smallest d_i ($O(n)$); (iv) output all a_i with d_i below that threshold, breaking ties arbitrarily ($O(n)$). Total $O(n)$. This was 2022 Q7.

4 Binary Search Trees

Problem — Dynamic dictionary

Binary search on a *sorted array* gives $O(\log n)$ search but $O(n)$ insertion. An unsorted array gives $O(1)$ insertion but $O(n)$ search. We want a structure supporting search, insert and delete efficiently under arbitrary interleaving.

Exam facts — definitions you must be able to state

- **Binary tree (recursive):** empty, or a root together with a left subtree and a right subtree, each itself a binary tree.
- **Height:** number of *edges* on the longest root-to-leaf path. (Some texts count vertices — watch for a problem that redefines it, and be consistent.)
- **BST property:** for every node v, all keys in v's left subtree are $< v$'s key, and all keys in v's right subtree are $> v$'s key.
- **Traversals:** Inorder = L, Root, R; Preorder = Root, L, R; Postorder = L, R, Root.
- **Inorder successor** of v: if v has a right subtree, the minimum of that subtree. **Inorder predecessor**: if v has a left subtree, the maximum of it.

Algorithm — Insert k

1. If the tree is empty, make k the root.
2. Else walk down from the root: go left if $k <$ current key, right if $k >$; on equality either skip or follow a fixed convention.
3. Insert at the first null child reached.

Algorithm — Delete node v with key k

Find v by search, then:

- **Case 1 — v is a leaf.** Remove it.
- **Case 2 — v has one child.** Splice v out: connect v's parent directly to v's only child.
- **Case 3 — v has two children.** Find v's inorder successor w (minimum of v's right subtree — necessarily has at most one child). Copy w's key into v, then delete w by Case 1 or 2. (The predecessor works equally, by symmetry.)

Correctness

Case 1 removes a node with no descendants — no ordering constraint is disturbed. Case 2 is valid because everything in v's subtree is on the same side of v's parent, so promoting the single child preserves that side's inequality. Case 3: w is the smallest key greater than k, so every key in v's left subtree is $< w$ and every remaining key in v's right subtree is $> w$ — exactly the BST property at v with the new key. Deleting w from the right subtree then falls under Case 1 or 2. ■

Time complexity

Search, insert and delete each walk one root-to-leaf path, so all cost $O(\text{height})$. In the worst case a BST degenerates to a path (insert already-sorted data), giving height $n-1$ and $O(n)$ per operation — **no better than an unsorted list**. This is precisely the motivation for balancing.

Exam facts — height-balanced (AVL) trees and the Fibonacci bound

A tree is **height-balanced** if for every node v, $\left|h(\text{left}(v)) - h(\text{right}(v))\right| \le 1$.
Minimum nodes at height h (vertex-counted): hang a minimal tree of height $h-1$ on one side and $h-2$ on the other (the largest imbalance still legal):

$$N(h) = 1 + N(h-1) + N(h-2), \qquad N(1) = 1,\ N(2) = 2.$$

This is Fibonacci-like, so $N(h)$ grows *exponentially* in h — which is exactly why $h = O(\log n)$. Worked value: $N(4) = 7$.
Maximum nodes at height h: a perfect binary tree, $M(h) = 2^h - 1$. So for a height-balanced tree with n nodes and height h, $n \in [N(h),\ 2^h - 1]$.
The balanced families:

- **AVL:** balance condition above; repaired after insert/delete by single or double **rotations** at the lowest unbalanced ancestor. $O(\log n)$ ancestors, $O(1)$ per rotation $\Rightarrow$ $O(\log n)$ rebalancing.
- **Red-black:** root black; no red node has a red child; every root-to-null path has the same number of black nodes. That last property forces $O(\log n)$ height (longest path $\le 2\times$ shortest, since reds cannot be consecutive). Cheaper rebalancing than AVL, slightly looser balance.
- **B-/B$^+$-trees:** many keys per node (order m: up to $m-1$ keys, m children), keeping the tree shallow to minimise *disk reads*. B$^+$-trees put all data in the leaves and chain them for range queries.

Exam facts — reconstructing a tree from traversals

Claim. Inorder together with *either* preorder or postorder uniquely determines a binary tree. Preorder + postorder does **not** (except when every node has 0 or 2 children).

Proof. (idea) With inorder + preorder: the first preorder element is the root; its position in the inorder sequence splits the rest into left and right subtrees; recurse. Same with postorder (root is last). Without inorder, a preorder "root, then one child" cannot reveal whether that child is a left or right child — genuinely ambiguous. ■

Special case worth remembering: for a *BST* with distinct keys, preorder *alone* suffices — because the inorder traversal is forced to be the sorted order.

5 Segmented Least Squares

Problem — Segmented least squares

Given $P = \{p_1, \ldots, p_n\}$ with $x_1 < x_2 < \cdots < x_n$ and a penalty $c > 0$: partition P into **contiguous** segments $\{p_i, \ldots, p_j\}$, fit each with its own least-squares line, and minimise

$$E + cL,$$

where E is the total squared error over all segments and L is the number of segments.

Why the penalty is needed. One line underfits data with several trends; one segment per pair of points drives E to 0 while grossly overfitting. The term cL buys the trade-off.

OLS recap. For a single segment, $y = ax + b$ with

$$a = \frac{\sum_i (y_i - \bar{y})(x_i - \bar{x})}{\sum_i (x_i - \bar{x})^2}, \qquad b = \bar{y} - a\bar{x}.$$

Algorithm — Dynamic programming

Let $e_{i,j}$ be the minimum squared error of a single line through $p_i, \ldots, p_j$, and let $\mathrm{opt}(j)$ be the optimal cost for $p_1, \ldots, p_j$, with $\mathrm{opt}(0) = 0$.

1. Precompute prefix sums of $\sum x$, $\sum y$, $\sum xy$, $\sum x^2$ in $O(n)$.
2. For all $1 \le i \le j \le n$, compute $e_{i,j}$ in $O(1)$ each from those prefix sums.
3. For $j = 1, \ldots, n$:
$$\mathrm{opt}(j) = \min_{1 \le i \le j} \Big(e_{i,j} + c + \mathrm{opt}(i-1) \Big),$$
recording the minimising i.
4. Answer $\mathrm{opt}(n)$; recover the partition by following the recorded indices back.

Correctness

Optimal substructure. In any optimal partition, the last point p_j lies in a single segment, which begins at some p_i. Given that segment, the remaining points $p_1, \ldots, p_{i-1}$ must themselves be partitioned optimally — otherwise substituting their optimum would strictly improve the whole solution, contradicting optimality. Since the true starting index i is unknown, minimising over all i considers the optimum among all possibilities, so the recurrence is exact. Induction on j then gives $\mathrm{opt}(j)$ correct for every j. ■

Time complexity

- **Naive:** each $e_{i,j}$ from scratch costs $O(n)$; there are $O(n^2)$ pairs, so $O(n^3)$ for the table, plus $O(n^2)$ for the DP $\Rightarrow O(n^3)$.
- **With prefix sums:** $O(n)$ preprocessing, then each $e_{i,j}$ in $O(1)$, so the table costs $O(n^2)$ and the DP costs $O(n^2)$ (it fills n entries, each a min over $\leq n$ choices).

$$\boxed{O(n^2) \text{ overall}} \qquad (\text{the } O(n^3) \text{ figure is only the naive route}).$$

Space: $O(n^2)$ for the table, reducible to $O(n)$ if $e_{i,j}$ is recomputed on the fly.

Exam facts — the DP template this instantiates

Every "partition a sequence optimally" problem in this course has the same shape:

$$\text{opt}(j) = \min_{i \leq j} \Big(\text{cost of segment } [i, j] + \text{penalty} + \text{opt}(i-1) \Big).$$

The only design work is (i) identifying the last segment as the thing to case on, and (ii) making the segment cost $O(1)$ via preprocessing. Say both explicitly for full marks.

6 Lower Bound for Comparison-Based Sorting

Problem — How fast can sorting be?

Show that any algorithm that sorts n elements using only pairwise comparisons needs $\Omega(n \log n)$ comparisons in the worst case.

Correctness

Decision-tree argument. Model the algorithm as a binary tree: each internal node is one comparison, each of the two branches is an outcome, and each leaf is an output permutation.

- There are $n!$ possible orderings of n distinct elements, and each must be reachable — otherwise some input is sorted incorrectly. So the tree has **at least $n!$ leaves**.
- A binary tree of depth d has at most 2^d leaves. Hence

$$2^d \geq n! \quad \Longrightarrow \quad d \geq \log_2(n!).$$

- By Stirling, $\log_2(n!) = \Theta(n \log n)$.

The worst-case number of comparisons is the depth d, so it is $\Omega(n \log n)$. ■

Equivalent phrasing: each comparison at best halves the set of permutations still consistent with what has been seen, $n! \to n!/2 \to n!/4 \to \cdots$, so $\log_2(n!)$ comparisons are needed before one permutation remains.

Exam facts — how this bound gets used

- Merge sort and heap sort achieve $O(n \log n)$, hence are **worst-case optimal** among comparison sorts: none can run in $o(n \log n)$.
- **Non-comparison sorts beat it** — counting sort, radix sort — but only under extra assumptions (bounded integer keys). If an exam problem says "$A[i] \in \{1, 2, 3, 4\}$" or "keys are integers in $[0, k)$", it is *inviting* counting sort, and there is no contradiction

with this theorem.

- **Reduction template (very examinable).** To prove some problem Π needs $\Omega(n \log n)$: show that an $O(T(n))$ algorithm for Π, plus $O(n)$ extra work, would sort. Then $T(n) = \Omega(n \log n)$.
  - **Convex hull.** Map $x_i \mapsto (x_i, x_i^2)$ onto the parabola. Every point is on the hull (the parabola is convex), and reading the hull off in order gives the x_i sorted. So convex hull is $\Omega(n \log n)$.
  - **Element uniqueness, closest pair** — same style.

7 3SUM and Reductions

Problem — 3SUM

Given a set S of n numbers, decide whether there exist $x, y, z \in S$ with $x + y + z = 0$.

Algorithm — Sort and two-pointer sweep — $O(n^2)$

1. Sort S in $O(n \log n)$.
2. For each fixed $x \in S$: look for $y + z = -x$ among the sorted elements with a two-pointer sweep — start a pointer ℓ at the smallest and r at the largest; if $S[\ell] + S[r]$ is too small increment ℓ, if too large decrement r, if equal report success. $O(n)$ per x.

Correctness

The sweep is correct because the array is sorted: if $S[\ell] + S[r] < -x$ then $S[\ell]$ cannot pair with anything at or below $S[r]$ to reach $-x$, so ℓ can be safely advanced (and symmetrically for r). No candidate pair is ever skipped, so if a valid (y, z) exists it is found. ■

Time complexity

n iterations of an $O(n)$ sweep, dominating the $O(n \log n)$ sort: $\boxed{O(n^2)}$.

Problem — Problem X (three-set variant)

Given sets A, B, C with $|A| + |B| + |C| = n$, decide whether $\exists\, a \in A, b \in B, c \in C$ with $a + b = c$.

Solved identically (sort, then two-pointer): $O(n^2)$.

Correctness

Claim. 3SUM $\leq_p X$ and $X \leq_p$ 3SUM, so they are equally hard.

3SUM $\leq_p X$**.** Given S, set

$$A = S, \qquad B = S, \qquad C = -S = \{-s : s \in S\}.$$

Then $a + b = c$ for some $a \in A, b \in B, c \in C \iff x + y = -z$ for some $x, y, z \in S \iff x + y + z = 0$. Construction is $O(n)$.

$X \leq_p$ 3SUM**.** Choose μ larger than $|v|$ for every $v \in A \cup B \cup C$, and set

$$A_S = \{a + \mu\}, \qquad B_S = \{b + 3\mu\}, \qquad C_S = \{-c - 4\mu\}.$$

Run 3SUM on $S = A_S \cup B_S \cup C_S$. For $u \in A_S$, $v \in B_S$, $w \in C_S$ arising from a, b, c:

$$u + v + w = (a + \mu) + (b + 3\mu) + (-c - 4\mu) = a + b - c,$$

which is 0 exactly when $a + b = c$. Because μ dominates every original number, the offsets $\mu, 3\mu, -4\mu$ are spread far enough apart that **a zero-sum triple can only take one element from each shifted set** — never two from the same one. So yes-instances correspond exactly. ■

Exam facts — what to say about 3SUM's status

- Whether 3SUM admits a *truly subquadratic* $O(n^{2-\epsilon})$ algorithm is a major open problem (the **3SUM conjecture**). Algorithms shaving polylog factors exist, e.g. $O(n^2/\text{polylog}\, n)$; nothing better is known.
- A problem is **3SUM-hard** if 3SUM reduces to it — so a subquadratic algorithm for it would break the conjecture. This is a *conditional* lower bound, not an unconditional one. Say "conditional" in the exam.
- The reduction direction matters: to show Π is 3SUM-hard you must reduce **3SUM** $\to \Pi$, not the other way.

8 3SUM-Hardness in Computational Geometry

Problem — Three collinear points

Given n points in $\mathbb{R}^2$, decide whether any three of them are collinear.

Algorithm — Slope sorting — $O(n^2 \log n)$

Checking all $\binom{n}{3}$ triples costs $O(n^3)$. Better:

1. For each point p_i: compute the slope from p_i to each of the other $n - 1$ points ($O(n)$), and sort those slopes ($O(n \log n)$).
2. If two of them are equal, those two points together with p_i are collinear.

Correctness

Three points are collinear iff, viewed from any one of them, the other two subtend the same slope. Fixing p_i and detecting a repeated slope therefore detects exactly the collinear triples containing p_i; ranging over all i covers every triple. ■

Time complexity

n points, each costing $O(n \log n)$ to sort slopes: $\boxed{O(n^2 \log n)}$.

Reduction 1: the cubic curve

Correctness

Claim. 3SUM $\leq_p$ {three collinear points}.

Proof. Given $x_1, \ldots, x_n$, map $x_i \mapsto (x_i, x_i^3)$ — all points onto the curve $y = x^3$ (an $O(n)$ construction). For distinct a, b, c the points $(a, a^3), (b, b^3), (c, c^3)$ are collinear iff the slope from a to c equals the slope from a to b:

$$\frac{c^3 - a^3}{c - a} = \frac{b^3 - a^3}{b - a} \iff c^2 + ca + a^2 = b^2 + ab + a^2 \iff c^2 - b^2 = a(b - c).$$

Factor the left side: $(c - b)(c + b) = -a(c - b)$. Since $b \neq c$ we may cancel $(c - b)$:

$$a + b + c = 0.$$

So the three points are collinear *iff* $a + b + c = 0$ — exactly the 3SUM condition. Hence any algorithm for three-collinear-points solves 3SUM at the same cost. ■

Reduction 2: three horizontal lines

Correctness

Claim. Problem X (hence 3SUM) $\leq_p$ {three collinear points}.

Construction. Given A, B, C, place

$$(a, 0) \ \ \forall a \in A, \qquad (b, 2) \ \ \forall b \in B, \qquad \left(\tfrac{c}{2}, 1\right) \ \ \forall c \in C.$$

So A sits on $y = 0$, B on $y = 2$, and C (halved) on $y = 1$ exactly in between. This is $O(|A| + |B| + |C|)$ time.

The collinearity computation. For $P_1 = (a, 0)$, $P_3 = (c/2, 1)$, $P_2 = (b, 2)$:

$$m_{13} = \frac{1 - 0}{\frac{c}{2} - a} = \frac{1}{\frac{c}{2} - a}, \qquad m_{32} = \frac{2 - 1}{b - \frac{c}{2}} = \frac{1}{b - \frac{c}{2}}.$$

Collinear $\iff m_{13} = m_{32} \iff \frac{c}{2} - a = b - \frac{c}{2} \iff c = a + b$.

Numeric check. $a = 1$, $b = 4$, so c should be 5. Points $(1, 0), (4, 2), (2.5, 1)$: slopes $\frac{1-0}{2.5-1} = \frac{1}{1.5} = \frac{2}{3}$ and $\frac{2-1}{4-2.5} = \frac{1}{1.5} = \frac{2}{3}$ — equal, collinear. ✓ With $c = 6$ instead (point $(3, 1)$) the slopes are $\frac{1}{2}$ and 1 — not collinear, matching $1 + 4 \neq 6$.

Why only "non-trivial" triples matter. Every point lies on one of the three horizontal lines $y = 0, 1, 2$. Two points on the same line already determine that horizontal line, so a third collinear point would have to lie on it too — impossible if it came from a different set. Hence *any* collinear triple mixing the sets takes exactly one point from each line. Triples inside a single line are trivially collinear and carry no information, so excluding them is exactly right. ■

Exam facts — on 3SUM-hard geometry

- Two independent reductions (cubic curve, three horizontal lines) give the same conclusion: **three-collinear-points is 3SUM-hard** (Gajentaan–Overmars).
- The gap is real: the best known algorithm is $O(n^2 \log n)$ (or $O(n^2)$ with duality/topological sweep), while the conditional lower bound is "no $O(n^{2-\epsilon})$".
- **Minimum-area triangle** (given n points, no three collinear, find the triangle of least

area) is the natural companion: brute force fixes a base pair ($\binom{n}{2}$) and scans the other $n-2$ points, giving $O(n^3)$. Whether it is 3SUM-hard is exactly the sort of open-ended question the notes flag.

9 Graphs: Representation and Breadth-First Search

Exam facts — the recipe, and how to store a graph

Recipe for every graph question: identify the property to compute or characterisation to test → design the algorithm → choose the data structure that makes it efficient.

Representation	Space	Test a specific edge
Adjacency matrix ($n \times n$ 0/1)	$\Theta(n^2)$ regardless of sparsity	$O(1)$
Adjacency list (neighbours per vertex)	$\Theta(\|V\| + \|E\|)$	$O(\deg)$

Use lists for sparse graphs (almost always in this course, since all the $O(|V| + |E|)$ bounds below assume lists).

Problem — BFS and its uses

Explore a graph outward from a source s in layers of increasing distance; use it to test connectivity, compute unweighted shortest paths, test bipartiteness, and find tree diameters.

Algorithm — Breadth-First Search

Three colours track progress: **white** (undiscovered), **grey** (discovered, in the queue), **black** (fully processed).

1. All vertices white. Colour s grey, enqueue it, set $d(s) = 0$.
2. While the queue is non-empty: pop u, colour it black; for every *white* neighbour v of u: colour v grey, set $d(v) = d(u) + 1$ and $\text{parent}(v) = u$, enqueue v.

Correctness

Claim. BFS discovers vertices in non-decreasing order of true shortest-path distance from s, and the parent pointers form a shortest-path tree.

Proof. Induction on the distance d. All vertices at distance 0 ($= s$) are handled first. Suppose every vertex at distance $< d$ has been discovered with the correct label, and that the queue processes them before anything farther. A vertex v at distance exactly d has some neighbour u at distance $d-1$; when u is dequeued, v is either still white (so it is discovered now with $d(v) = d(u) + 1 = d$) or was already discovered — and it cannot have been discovered from a vertex at distance $< d-1$, since that would give v distance $< d$. So v is labelled exactly d. The FIFO queue guarantees layer $d-1$ is fully processed before layer d begins. ■

Time complexity

Every vertex is enqueued and dequeued exactly once ($O(|V|)$); every edge is examined at most twice, once from each endpoint ($O(|E|)$). Total $\boxed{O(|V| + |E|)}$.

Trees: diameter and farthest vertex

Problem — Diameter of a tree

Given a tree T, compute $\text{diam}(T) = \max_{u,v} d(u, v)$.

Algorithm — The two-BFS trick (trees only)

1. BFS from an arbitrary vertex s; let u be a vertex farthest from s.
2. BFS from u; let v be a vertex farthest from u.
3. Return $d(u, v)$.

Correctness

Key fact. In a tree, for *any* vertex s, a vertex farthest from s is an endpoint of some diametral pair.
Granting this: u is farthest from the arbitrary s, so u pairs with some b having $d(u, b) = \text{diam}(T)$. Since v is farthest from u, $d(u, v) \geq d(u, b) = \text{diam}(T)$. But no pairwise distance can exceed the diameter, so $d(u, v) = \text{diam}(T)$ exactly. ■

Time complexity

Two BFS runs: $\boxed{O(|V| + |E|)} = O(n)$ for a tree.

Trap — the two-BFS trick is *special to trees*

It is **not valid** for general graphs. For a general graph, computing the exact diameter essentially requires BFS from every vertex — $O(|V|(|V| + |E|))$.
Also: to find the vertex farthest from a given v, use **BFS, not DFS**. DFS goes deep down one branch and may report a vertex that is deep in the DFS tree but not farthest in true shortest-path distance.

Exam facts — recursive diameter, and eccentricities in $O(n)$

Recursive formulation. For a subtree rooted at v whose children lead to subtrees with (diameter, depth) pairs $(d_1, h_1), (d_2, h_2), \ldots$, the diameter of the whole is the larger of

- $\max_i d_i$ (the path avoids v), and
- (sum of the two largest h_i) $+2$ (the path passes through v between its two deepest branches).

All eccentricities at once. If (u, v) is a diametral pair, then for every vertex x,

$$\text{ecc}(x) = \max\{d(x, u),\ d(x, v)\}.$$

So two more BFS runs (from u and from v) give every eccentricity in $O(n)$ total.

Edge classification, DAGs, topological sort

Exam facts — edge types relative to a search tree

- **Tree edge** — used to first discover a vertex.
- **Back edge** — to an ancestor in the tree.
- **Forward edge** — to a non-child descendant.
- **Cross edge** — between vertices with no ancestor/descendant relation.

In a BFS tree of an *undirected* graph, only tree and cross edges occur, and cross edges join vertices at the same or adjacent levels. Forward and back edges cannot appear. In *directed* graphs all four types are possible.

Problem — Topological sort

A **DAG** is a directed graph with no directed cycle. A **topological order** lists the vertices so that for every edge (u, v), u precedes v. Compute one, or report that none exists.

Algorithm — Two standard methods

Kahn's algorithm. Repeatedly output a vertex of in-degree 0 and delete it with its outgoing edges. If vertices remain but none has in-degree 0, the graph has a cycle and no topological order exists.
DFS-based. Run DFS on the whole graph and output vertices in *decreasing order of finishing time*.

Correctness

Kahn. A vertex of in-degree 0 has no constraint forcing anything before it, so outputting it first is safe; deleting it leaves a DAG, and induction finishes. Conversely, if no in-degree-0 vertex exists in a non-empty digraph, follow in-edges backwards — with finitely many vertices you must repeat one, exhibiting a cycle.
DFS. For every edge (u, v): if v is white when the edge is explored, v finishes before u; if v is black, it already finished before u; v cannot be grey, since that would make (u, v) a back edge, i.e. a cycle. So in all cases $f(v) < f(u)$, and sorting by decreasing finish time puts u before v. ■

Time complexity

Both are $\boxed{O(|V| + |E|)}$ (Kahn with an in-degree array and a queue of zero-in-degree vertices).

10 Planar Graphs and Graph Colouring

Exam facts — planarity and Euler's formula

G is **planar** if it can be drawn in the plane with no two edges crossing.
Kuratowski (1930). G is planar $\iff$ it contains no subgraph that is a subdivision of K_5 or $K_{3,3}$. (Equivalently, by Wagner, no K_5 or $K_{3,3}$ *minor*.)
Euler's formula. For a *connected* planar graph drawn in the plane with V vertices, E edges

and F faces (counting the unbounded outer face),

$$\boxed{V + F - E = 2.}$$

Testing planarity is $O(|V|)$ (Hopcroft–Tarjan), built from an incremental DFS embedding — *not* by searching for forbidden subgraphs. Know it by name.

Correctness

Claim (sparsity). Every simple planar graph with $n \geq 3$ vertices has $E \leq 3n - 6$.

Proof. Every face is bounded by at least 3 edges, and every edge borders exactly 2 faces, so counting incidences two ways gives $2E \geq 3F$, i.e. $F \leq \frac{2}{3}E$. Substituting into Euler:

$$2 = n - E + F \leq n - E + \tfrac{2}{3}E = n - \tfrac{1}{3}E \quad \Longrightarrow \quad \tfrac{1}{3}E \leq n - 2 \quad \Longrightarrow \quad E \leq 3n - 6. \qquad \square$$

Claim (min degree). Every planar graph has a vertex of degree ≤ 5.

Proof. Suppose every vertex had degree ≥ 6. Then $2E = \sum_v \deg(v) \geq 6n$, so $E \geq 3n$. But planarity forces $E \leq 3n - 6$, giving $3n \leq 3n - 6$ — a contradiction. ■

Problem — Colour a planar graph

Assign colours to vertices so adjacent vertices differ, using as few colours as possible. $\chi(G)$ denotes the minimum.

Correctness

Six-Colour Theorem. Every planar graph is 6-colourable.

Proof. Induction on $n = |V|$. Trivial for $n \leq 6$. For larger n: by the min-degree claim there is a vertex v with $\deg(v) \leq 5$. Delete v; the rest is still planar with fewer vertices, so by induction it is 6-colourable. Restore v: its ≤ 5 neighbours use at most 5 of the 6 colours, so a colour remains free for v. ■

Algorithm — 6-colouring, constructively

1. Repeatedly remove a minimum-degree vertex (always ≤ 5), recording the removal order $v_n, v_{n-1}, \ldots, v_1$ (so v_n was removed first).
2. Colour in **reverse** of the removal order: give v_1 any colour; for each subsequent v_i, assign the lowest-indexed colour unused by its already-coloured neighbours — of which there are at most 5.

Time complexity

Step 1 is a sequence of extract-min and decrease-key operations on vertex degrees, supported by a binary heap or balanced BST:

$$\boxed{O((|V| + |E|) \log |V|)}.$$

(With bucket-by-degree instead of a heap this drops to $O(|V| + |E|)$, since degrees are small integers.) Step 2 is $O(|V| + |E|)$.

Correctness

Five-Colour Theorem. Every planar graph is 5-colourable.

Proof. (sketch — the Kempe chain) Induct as before: remove v with $\deg(v) \le 5$, 5-colour the rest, restore v.

- If v's neighbours use ≤ 4 distinct colours (either $\deg(v) < 5$, or two neighbours happen to share a colour), a fifth colour is free — done.
- **Hard case:** $\deg(v) = 5$ with neighbours $v_1, \dots, v_5$ in cyclic order around v (planarity gives the cyclic order), coloured with 5 distinct colours $c_1, \dots, c_5$.

Consider the subgraph induced by vertices coloured c_1 or c_3.

- If v_1 and v_3 lie in *different* components of it, swap $c_1 \leftrightarrow c_3$ throughout v_1's component. Still a proper colouring, and now v_1 is c_3, freeing c_1 for v.
- If they lie in the *same* component, there is a c_1/c_3-alternating path from v_1 to v_3. That path plus the edges vv_1 and vv_3 forms a closed curve in the plane which **separates** v_2 **from** v_4. Any c_2/c_4-alternating path from v_2 to v_4 would have to cross it — forbidden by planarity. So v_2, v_4 lie in different c_2/c_4-components, and we swap in v_2's component instead, freeing c_2.

Either way a colour is freed. ■

Exam facts — four colours, edge colouring, and bipartiteness

- **Four-Colour Theorem.** Every planar graph is 4-colourable (Appel–Haken, 1976), but every known proof needs a computer-assisted check of a large finite family of configurations. **It was not proved in this course** — only 5 and 6 were. If asked to *prove* a colouring bound for planar graphs, prove 6 (short) or 5 (Kempe chain); never claim to prove 4.
- **Edge colouring.** Colour edges so that edges sharing an endpoint differ; the minimum is the chromatic index $\chi'(G)$. Trivially $\chi'(G) \ge \Delta(G)$. **Vizing:** $\chi'(G) \in \{\Delta(G), \Delta(G)+1\}$ — always within one of the trivial bound, unlike vertex colouring where χ can be far above ω.
- **Bipartiteness.** G (with at least one edge) is bipartite $\iff \chi(G) = 2 \iff G$ has no odd cycle.

Algorithm — Bipartiteness test via BFS — $O(|V| + |E|)$

Run BFS from any vertex, colouring by level parity (level 0 Red, level 1 Black, level 2 Red, …). If any edge is found joining two vertices at the *same* level, report "not bipartite"; otherwise report "bipartite". Repeat per connected component.

Correctness

If no edge joins two same-level vertices, then every edge joins consecutive levels, so the level-parity colouring is a proper 2-colouring and G is bipartite.
Conversely, every BFS *tree* edge joins consecutive levels by construction, and a non-tree edge cannot join levels differing by more than 1 (the shallower endpoint would have discovered the deeper one first). So the *only* possible failure is a non-tree edge inside one level — which is exactly what the test looks for. Such an edge creates an odd cycle (two equal-length tree paths to the common ancestor, plus the edge). ■

11 Independent Sets, Cliques and Perfect Graphs

Exam facts — the four parameters

- **Independent set:** no two vertices adjacent. *Maximal* = cannot be extended; **maximum** = largest possible. **These differ** — a maximal independent set need not be maximum. $\alpha(G)$ is the maximum size.
- $\omega(G)$ — **clique number**, size of the largest complete subgraph.
- $\chi(G)$ — **chromatic number**, fewest colours in a proper colouring.
- $\kappa(G)$ — **clique cover number**, fewest cliques partitioning $V(G)$.

Correctness

Claim. $\alpha(G) = \omega(\overline{G})$.

Proof. A set S has no two vertices adjacent in $G \iff$ every pair in S is adjacent in $\overline{G}$. So independent sets of G are exactly the cliques of $\overline{G}$, and the largest of each coincide. ■

Claim. $\kappa(G) = \chi(\overline{G})$.

Proof. A proper colouring of $\overline{G}$ partitions V into colour classes, each independent in $\overline{G}$, i.e. each a clique in G. Minimising the number of classes is therefore the same optimisation as minimising the number of cliques covering G. ■

Claim. $\chi(G) \geq \omega(G)$, and the inequality can be strict.

Proof. Every vertex of a clique needs its own colour, so $\chi \geq \omega$. Strictness: for an odd cycle C_{2k+1} with $k \geq 2$ (length ≥ 5), $\omega = 2$ (no triangle) but $\chi = 3$ (odd cycles are not bipartite). ■

Exam facts — perfect graphs — and the definition trap

Definition. G is **perfect** if $\chi(H) = \omega(H)$ for **every induced subgraph** H of G — *not* merely for G itself.

Trap — $\chi(G) = \omega(G)$ at the top level is *not* enough

Let $G = C_5 \sqcup K_3$ (disjoint union). Then

$$\chi(G) = \max(3, 3) = 3, \qquad \omega(G) = \max(2, 3) = 3,$$

so $\chi(G) = \omega(G)$ — yet the induced subgraph C_5 alone has $\chi = 3 \neq \omega = 2$, so G is **not** perfect. Always check induced subgraphs.

Basic non-example: C_5 itself is not perfect, since $\chi(C_5) = 3 \neq 2 = \omega(C_5)$.

Strong Perfect Graph Theorem (Chudnovsky–Robertson–Seymour–Thomas, 2006): G is perfect $\iff$ G contains no induced *odd hole* (induced odd cycle of length ≥ 5) and no induced *odd antihole* (complement of one).

12 Intersection, Interval and Chordal Graphs

Problem — Intersection graphs

Given a family of finite sets, one per vertex, put an edge between two vertices exactly when their sets intersect. The resulting graph is the **intersection graph** of that family.

Correctness

Marczewski's Theorem. *Every* graph is the intersection graph of some family of sets.

Proof. Given $G(V, E)$, for each vertex v_i set

$$S(v_i) = \{v_i\} \cup \{e : e \text{ is an edge incident to } v_i\}.$$

For $i \neq j$, the only element that $S(v_i)$ and $S(v_j)$ could possibly share is the edge label $\{v_i, v_j\}$ (vertex names are unique, and any other edge label touches at most one of them). That label lies in both sets exactly when $v_iv_j \in E$. So $S(v_i) \cap S(v_j) \neq \emptyset \iff v_iv_j \in E$. ■

Exam facts — intersection number — and a discrepancy to be careful about

The **intersection number** of G is the smallest ground set (union of all $S(v)$) admitting such a representation. The classical characterisation:

intersection number = minimum number of cliques needed to cover every *edge* (an *edge clique cover*),

— put one ground-set element per clique, and place it in $S(v)$ for every v in that clique.

Trees. For a tree T on n vertices, the intersection number is $|E(T)| = n - 1$.

Proof. A tree is triangle-free, so its only edge-covering cliques are single edges; no clique covers two edges at once. Hence exactly $|E(T)| = n - 1$ cliques are needed. ■

Trap: "$n - \#$leaves" is wrong. On $P_4 = v_1v_2v_3v_4$ that guess gives 2, but with a 2-element ground set there are only 3 non-empty subsets, and casework shows no assignment realises the 3 required edges while avoiding the 3 required non-edges. The correct value is $n - 1 = 3$. (The guess coincidentally matches on a star, which is why it can look right.)

K_n: two different answers depending on the phrasing — read the question.

- **Sets need not be distinct** (the standard definition, matching the edge-clique-cover characterisation): one clique — all of V — covers every edge, so the intersection number of K_n is $\boxed{1}$: take $S(v) = \{1\}$ for every v.
- **Sets required to be distinct** (the variant worked in the lecture notes): use only subsets containing one fixed element, so any two automatically intersect. There are 2^{m-1} such subsets of an m-element ground set, so we need $2^{m-1} \geq n$, i.e.

  $$m = \lceil \log_2 n \rceil + 1.$$

  Check: $n = 4 \Rightarrow m = 3$; $n = 5 \Rightarrow m = 4$; $n = 8 \Rightarrow m = 4$ — matching the notes' worked numbers. So this variant is $\Theta(\log n)$.

Either way the contrast with trees is the point: trees are the expensive extreme $(n - 1)$, complete graphs the cheap one.

Problem — Interval graphs

Represent each vertex by an interval on the real line, with an edge exactly when two intervals overlap.

Exam facts — what is and is not an interval graph

- K_3 **is** one (three mutually overlapping segments).
- C_4 is **not**. Label the cycle $v_1v_2v_3v_4$: v_3 must overlap both v_2 and v_4 but not v_1. Since v_2 and v_4 must sit on either side of v_1's interval without touching each other, anything overlapping both of them must also overlap v_1 — contradiction.
- **Not every tree** is one. Take a subdivided claw (a spider with three legs each of length ≥ 2) and let v_1, v_5, v_6 be the three leg-ends. The tree requires a path between each pair avoiding the third — an **asteroidal triple** — which three intervals on a line cannot realise, whatever their order.
- **Lekkerkerker–Boland.** G is an interval graph $\iff$ G is chordal and has no asteroidal triple. For trees this specialises to: **a tree is an interval graph $\iff$ it is a caterpillar** (no vertex has ≥ 3 non-leaf neighbours).

Problem — Chordal graphs

G is **chordal** if it has no induced cycle of length ≥ 4; equivalently every cycle of length ≥ 4 has a chord.

Exam facts — the containment chain

$$\text{interval} \subset \text{chordal} \subset \text{perfect}.$$

So interval graphs are perfect too. **Contrast:** disc graphs (intersection graphs of discs in the plane) are *not* all perfect — five overlapping discs in a ring realise an induced C_5, which is not perfect.

Problem — Perfect elimination ordering (PEO)

An ordering $v_1, \ldots, v_n$ of $V(G)$ is a **PEO** if for every i, the set $N(v_i) \cap \{v_1, \ldots, v_{i-1}\}$ (the *earlier* neighbours of v_i) forms a clique.

Correctness

Claim. If G has a PEO then G is chordal.

Proof. Suppose not: let $v_{i_1}, \ldots, v_{i_k}$ be an induced (chordless) cycle with $k \geq 4$. Put $M = \max\{i_1, \ldots, i_k\}$, so v_M is the *last* cycle vertex in the ordering. Its two cycle-neighbours v_p, v_q both come earlier and are both adjacent to v_M, so both lie in $N(v_M) \cap \{v_1, \ldots, v_{M-1}\}$ — which the PEO property says is a clique. Hence v_pv_q is an edge. But v_p and v_q are non-consecutive on the cycle (they are the two neighbours of v_M, and $k \geq 4$), so that edge is a **chord** — contradicting chordlessness. ■

Converse (sketch). Every chordal graph has a PEO. Every chordal graph has a **simplicial vertex** (one whose entire neighbourhood is a clique); deleting it leaves a chordal graph (deletion cannot create a chordless cycle), so induction builds the ordering one vertex at a time. Constructively, Lex-BFS produces such an ordering.

Trap — greedy colouring depends on the order

Colouring greedily (each vertex gets the lowest colour unused by its already-coloured neighbours) in an *arbitrary* order can use more than $\chi(G)$ colours — e.g. a 5-vertex graph with $\chi = 3$ can be forced to 4 by a bad ordering. This is exactly why one wants a *good* ordering, which is what a PEO provides.

Exam facts — why a PEO is so useful — $\chi = \omega$ for chordal graphs

Colour greedily *along* the PEO $v_1, v_2, \ldots, v_n$. When v_i is coloured, its earlier neighbours form a clique of some size d_i, so they use exactly d_i distinct colours and v_i takes colour number $\leq d_i + 1$. Hence

$$\chi(G) \;\leq\; 1 + \max_i d_i.$$

But $\{v_i\} \cup \big(N(v_i) \cap \{v_1, \ldots, v_{i-1}\}\big)$ is a clique of size $d_i + 1$, so $\omega(G) \geq 1 + \max_i d_i$. Combined with the universal $\chi \geq \omega$:

$$\boxed{\chi(G) = \omega(G) = 1 + \max_i d_i \quad \text{for chordal } G,}$$

computable in $O(|V| + |E|)$ once the PEO is known. This is the constructive reason chordal graphs are perfect.

13 Lex-BFS: Worked Exercise

Problem — The exercise

On the graph $V = \{1, 2, 3, 4, 5\}$ with

$$E = \{12,\ 15,\ 13,\ 14,\ 34,\ 35,\ 45,\ 25\},$$

(i) state Lex-BFS rigorously, (ii) run it and verify the output is a PEO, (iii) use the PEO to build a maximum independent set.

Structure of the graph. $\{1, 3, 4, 5\}$ induces a K_4 (all six pairs $13, 14, 15, 34, 35, 45$ are edges), and vertex 2 attaches only to 1 and 5 (so $\{1, 2, 5\}$ is a triangle). Degrees: $\deg(1) = \deg(5) = 4$, $\deg(3) = \deg(4) = 3$, $\deg(2) = 2$.

Algorithm — Lex-BFS

Each vertex carries a **label**: a sequence of round numbers, most recent first, initially empty. Labels compare lexicographically entry by entry (a larger first entry beats a smaller or absent one).

1. For every v: label$(v) \leftarrow ()$. Let $U \leftarrow V(G)$ and order $\leftarrow ()$.
2. For $k = 1, 2, \ldots, n$:
   (a) Choose $v \in U$ with lexicographically *largest* label (break ties by a fixed rule, e.g. smallest index).
   (b) Append v to order as v_k; remove v from U.
   (c) For every $u \in U \cap N(v)$: **prepend** k to label(u).
3. Return order $= (v_1, \ldots, v_n)$.

Time complexity

Naively, scanning all of U each round to find the lex-largest label costs $O(n^2)$. With **partition refinement** (bucket vertices by label so the current maximum is $O(1)$ to find, and split buckets incrementally) it is $\boxed{O(|V| + |E|)}$.

Running it

Round k	Pick v_k	Unvisited nbrs updated	Labels after
1	1 (all tied at ())	$2, 3, 4, 5$	$\ell(2)$=$\ell(3)$=$\ell(4)$=$\ell(5) = (1)$
2	2 (tie at (1), smallest index)	5	$\ell(5) = (2, 1)$; $\ell(3)$=$\ell(4) = (1)$
3	5 ($(2, 1) \succ (1)$)	$3, 4$	$\ell(3) = \ell(4) = (3, 1)$
4	4 (genuine tie at $(3, 1)$)	3	$\ell(3) = (4, 3, 1)$
5	3 (only one left)	—	—

Output: $(v_1, \ldots, v_5) = (1, 2, 5, 4, 3)$.

Correctness

Verifying it is a PEO. Check $N(v_i) \cap \{v_1, \ldots, v_{i-1}\}$ is a clique at each step:

i	v_i	$N(v_i) \cap \{\text{earlier}\}$	Clique?
1	1	$\emptyset$	trivially
2	2	$\{1\}$	single vertex ✓
3	5	$\{1, 2\}$	$12 \in E$ ✓
4	4	$\{1, 5\}$	$15 \in E$ ✓
5	3	$\{1, 4, 5\}$	$14, 15, 45 \in E$ ✓

Every step checks out, so $(1, 2, 5, 4, 3)$ is a valid PEO — and hence the graph is chordal.

Exam facts — key fact: simplicial vertices and maximum independent sets

Claim. If v is simplicial in G (i.e. $N(v)$ is a clique), then *some* maximum independent set of G contains v.

Proof. Let S be any maximum independent set. If $v \in S$, done. Otherwise, since $N(v)$ is a clique and S is independent, $|S \cap N(v)| \leq 1$.

- If $S \cap N(v) = \emptyset$, then $S \cup \{v\}$ is independent and strictly larger — which contradicts S being maximum. So this cannot happen.
- So $S \cap N(v) = \{u\}$ for exactly one u. Then $S' = (S \setminus \{u\}) \cup \{v\}$ is independent (v is adjacent to nothing else in S, since u was its only neighbour there) and $|S'| = |S|$. So S' is also maximum, and contains v. ■

Algorithm — Maximum independent set on a chordal graph

1. Compute a PEO by Lex-BFS.
2. **Reverse** it to get an elimination order (each vertex simplicial in the graph remaining when it is processed).
3. Scan in that order, greedily adding v to I if none of its neighbours is already in I, then marking $N(v)$ unavailable.

Correctness

By the key fact applied inductively — each time to the graph induced on the not-yet-processed vertices — some maximum independent set is consistent with every greedy choice made. Hence the final I is maximum. ■

Running the greedy on our example

Reversing the PEO $(1, 2, 5, 4, 3)$ gives the elimination order $(3, 4, 5, 2, 1)$. Sanity-check simpliciality: $N(3) = \{1, 4, 5\}$ is a clique ✓; in $G-3$, $N(4) \cap \{1, 2, 5\} = \{1, 5\}$ ✓; in $G-\{3, 4\}$, $N(5) \cap \{1, 2\} = \{1, 2\}$ ✓.

Vertex	Action	I
3	available → add; mark $N(3) = \{1, 4, 5\}$	$\{3\}$
4	marked → skip	$\{3\}$
5	marked → skip	$\{3\}$
2	available → add; mark $N(2) = \{1, 5\}$	$\{3, 2\}$
1	marked → skip	$\{2, 3\}$

Result: $I = \{2, 3\}$, size 2 (indeed $23 \notin E$). **Genuinely maximum:** any independent set contains at most one vertex of the clique $\{1, 3, 4, 5\}$, and vertex 2 can be added on top only if that vertex is 3 or 4 (since 2 is adjacent to 1 and 5). So 2 is the best possible — no independent set of size ≥ 3 exists.

14 What the 2022 paper actually tested

The last time this professor set this paper (22 September 2022; 60 marks, 2 h 30 m, "you may answer any part of any question, but maximum you can score is 60"), every question was an **algorithm-design** question. Here is the mapping onto this crash course:

2022 Q	Topic	Technique	Here
1(a)	Sort $A[i] \in \{1, 2, 3, 4\}$ in $O(n)$	counting sort	§6 facts
1(b)	In-place rearrange by a 0/1 key	two-pointer partition	§3 (partition step)
2(a)	Convex hull in $O(n \log n)$	sort + scan	§6 facts
2(b)	Convex hull is $\Omega(n \log n)$	reduction from sorting	§6
3(a)	$\mathbb{E}$(#inversions)	linearity of expectation	Practice Q3
3(b)	$T(n) = aT(n/b) + cn^k$, $a > b^k$	unroll the geometric sum	§2
4	Unit interval with most points	two-pointer sweep	Practice Q4
5	Subset sum closest to x from below	DP	§5 template
6	Count pairs $A[i] > 2A[j]$ in $O(n \log n)$	merge-sort variant	Practice Q5
7	k numbers nearest the median in $O(n)$	median of medians	§3 facts

Exam facts — what this tells you about the exam

- **Nothing was pure bookwork.** Even the two theory parts (2(b), 3(b)) were *proofs*, not statements.
- **Every algorithm question named a target complexity.** That target is the hint: $O(n)$ means counting/two-pointer/selection; $O(n \log n)$ means sort-then-scan or a merge-sort variant; $O(n^2)$ means DP or all-pairs.
- **The 2022 paper contained no graph theory** — but §§9–13 of your notes are entirely graph theory, so the syllabus has clearly widened. Prepare both. Practice Q7–Q9 below cover that half.
- **Always give all four parts:** algorithm, correctness, complexity, and the data structure that achieves it. The marks are split across them.

15 The 2022 Mid-Semester Paper

INDIAN STATISTICAL INSTITUTE
Mid Semestral Examination
B.Stat. — 3rd Year (Semester V)
Design and Analysis of Algorithms
Date: September 22, 2022 Maximum Marks: 60 Duration: 2:30 Hours
Note: You may answer any part of any question, but maximum you can score is 60.

Q1. [**5+5** marks]

(a) A is an unsorted array of size n where $A[i] \in \{1, 2, 3, 4\}$ for each $i \in \{0, 1, 2, \ldots, n-1\}$. Write an algorithm to sort the array A in $O(n)$ time.

(b) Let $S = \langle (a_i, b_i) : a \in \mathbb{N},\ b \in \{0, 1\} \rangle$ be a sequence stored in an array of size $2n$. Write a linear-time algorithm using $O(1)$ space that rearranges S such that all the tuples (a_i, b_i) having $b_i = 0$ appear at the beginning.

Q2. [**7+3** marks]

(a) Let S be a point set in the plane where $|S| = n$. Design an algorithm to find the convex hull of S in $O(n \log n)$ time.

(b) Prove that the lower bound of an algorithm that finds the convex hull of a given set of points S is $\Omega(n \log n)$, where $|S| = n$.

Q3. [**5+5** marks]

(a) An inversion in an array A is a pair (i, j) having the property $i < j$ and $A[i] > A[j]$. Let X be the total number of inversions in an array. Calculate $\mathbb{E}(X)$.

(b) Let $T(n)$ be a positive and non-decreasing function such that $T(n) = a\,T(n/b) + c\,n^k$ and $T(1) = d$ where $a, c > 0$, $k, d \geq 0$, $b \geq 2$. Prove that $T(n) = \Theta(n^{\log_b a})$.[1]

Q4. [**10** marks]

Let $S = \langle a_1, a_2, \ldots, a_n \rangle$ be an increasing sequence of points on the real line. Write a linear time algorithm that determines a unit length interval that contains maximum number of points of S.

Q5. [**10** marks]

Let S be a set of size n and x be a number. The sum of the elements of the set S is m. Write an efficient algorithm to determine the subset of S whose sum is closest to x but does not exceed x.

Q6. [**10** marks]

A pair (i, j) is called a *significant pair* if $i < j$ and $A[i] > 2A[j]$. Design an $O(n \log n)$-time algorithm to count the number of significant pairs in an array.

Q7. [**10** marks]

Let S be a set of n distinct numbers and k be a positive integer such that $k \leq n$. Write an $O(n)$-time algorithm that determines the k numbers in S that are nearest to the median of S.

16 Solutions to the 2022 Paper

Q1. [**5+5** marks]

(a) Counting sort with four counters.

[1] The scanned paper carries a handwritten addition "*and* $a > b^k$" — without it the claim is false (see §2), so read the condition as part of the question.

1. Initialise $C[1..4] \leftarrow 0$.
2. One pass: for each i, increment $C[A[i]]$.
3. One pass: write $C[1]$ copies of 1, then $C[2]$ copies of 2, then $C[3]$ copies of 3, then $C[4]$ copies of 4, overwriting A from the front.

Correctness. The output is non-decreasing by construction, and it is a permutation of the input because each value v is written exactly $C[v] = \#\{i : A[i] = v\}$ times.

Complexity. Two linear passes plus $O(1)$ counter work: $\boxed{O(n)}$ time, $O(1)$ extra space (four counters).

Trap — "but sorting is $\Omega(n \log n)$!"

No contradiction. The $\Omega(n \log n)$ bound of §6 applies only to **comparison-based** algorithms. Counting sort never compares two array elements — it uses their *values as indices*. Say this sentence explicitly; it is almost certainly worth a mark.

(b) Two-pointer (Lomuto) partition on the b-field.

1. $i \leftarrow 0$.
2. For $j = 0, 1, \ldots, n-1$: if $b_j = 0$, swap the tuples at positions i and j, then $i \leftarrow i + 1$.

(If the array literally stores the flattened sequence $a_1, b_1, a_2, b_2, \ldots$ in $2n$ cells, a "swap" is a swap of two adjacent-cell blocks — still $O(1)$ per operation.)

Loop invariant. Before each iteration, positions $[0, i)$ hold exactly the tuples with $b = 0$ seen so far, and positions $[i, j)$ hold exactly those with $b = 1$ seen so far.

Correctness. If $b_j = 1$ the invariant extends trivially (the region of 1s grows by one). If $b_j = 0$, the swap sends the tuple to position i — the first slot of the 1s block — and moves that displaced 1-tuple to position j, which is the new end of the 1s block; then i advances. Both blocks stay correct. On termination $j = n$, so $[0, i)$ is all the $b = 0$ tuples and $[i, n)$ all the $b = 1$ tuples.

Complexity. n iterations, each $O(1)$: $\boxed{O(n)}$ time, $O(1)$ space (two indices).

Remark. This is exactly the partition primitive of quicksort/quickselect, and the same idea as the PARTITION step of median-of-medians (§3). The output is **not stable**; if stability were demanded, $O(1)$ space and linear time together become genuinely hard.

Q2. [**7+3** marks]

(a) Andrew's monotone chain.

1. Sort the points lexicographically by (x, y).
2. **Lower hull:** scan left to right with a stack; before pushing p, while the top two stack points q, r and p fail to make a counter-clockwise turn — i.e. the cross product $(r - q) \times (p - q) \leq 0$ — pop the top.
3. **Upper hull:** run the same scan on the points in reverse order.
4. Concatenate the two chains, dropping the two duplicated endpoints.

(Graham scan is an equally acceptable answer: pick the lowest point, sort the rest by polar angle about it, then run the same left-turn stack scan.)

Correctness. A point is a vertex of the lower hull exactly when it makes a strict left turn with its two hull neighbours. The stack maintains the invariant "*the stack currently holds the lower hull of the points scanned so far*". Popping removes precisely those points that the new point p renders non-convex, and a popped point can never return: it lies strictly below the segment joining two points that remain, so it is inside the hull of the points already seen, and adding more points only enlarges that hull. Doing the scan in both directions yields the lower and upper chains, whose concatenation is the full hull.

Complexity. Sorting is $O(n \log n)$. In each scan every point is pushed once and popped at most once, so each scan is $O(n)$ amortised. Total $\boxed{O(n \log n)}$ — which by part (b) is optimal.

(b) Lower bound by reduction from sorting.

Let $\mathcal{H}$ be any algorithm computing the convex hull (as a cyclically ordered list of vertices) in time $T(n)$. Given arbitrary reals $x_1, \ldots, x_n$ to sort:

1. In $O(n)$ time build the point set $P = \{(x_i, x_i^2) : i = 1, \ldots, n\}$, i.e. lift every number onto the parabola $y = x^2$.
2. Run $\mathcal{H}$ on P, costing $T(n)$.
3. In $O(n)$ time, read the output starting from the leftmost vertex and walking along the lower chain, outputting the x-coordinates.

Why step 3 works. The parabola $y = x^2$ is **strictly convex**, so *every* point of P is a vertex of the hull — no point lies inside or on a chord joining two others. The hull is therefore a convex polygon on all n points, and traversing its boundary from the leftmost vertex along the lower chain visits them in strictly increasing order of x. So step 3 emits $x_1, \ldots, x_n$ sorted.

Hence sorting n reals is possible in $T(n) + O(n)$ time. But comparison-based sorting requires $\Omega(n \log n)$ (§6, decision-tree argument), so

$$T(n) + O(n) = \Omega(n \log n) \quad \Longrightarrow \quad \boxed{T(n) = \Omega(n \log n)}. \qquad \square$$

Exam facts — the reduction template, in three lines

To prove problem Π needs $\Omega(n \log n)$: **(i)** transform a sorting instance into a Π instance in $O(n)$; **(ii)** show a solution to Π can be converted back into the sorted order in $O(n)$; **(iii)** conclude $T_\Pi(n) + O(n) = \Omega(n \log n)$. State all three steps — marks are awarded per step, and step (ii) is the one people omit.

Q3. [**5+5** marks]

(a) The statement is only meaningful once the array is random, so take A to be a **uniformly random permutation** of n distinct numbers.

For each pair $i < j$ define the indicator

$$X_{ij} = \mathbf{1}\{A[i] > A[j]\}, \qquad \text{so} \qquad X = \sum_{1 \le i < j \le n} X_{ij}.$$

Fix such a pair. In a uniformly random permutation the two relative orders of the values at positions i and j are equally likely, so $\mathbb{P}(A[i] > A[j]) = \frac{1}{2}$ and $\mathbb{E}(X_{ij}) = \frac{1}{2}$.

Linearity of expectation needs *no* independence (and indeed the X_{ij} are highly dependent), so

$$\mathbb{E}(X) = \sum_{i<j} \mathbb{E}(X_{ij}) = \binom{n}{2} \cdot \frac{1}{2} = \boxed{\frac{n(n-1)}{4}}.$$

Sanity checks. $n = 2$ gives $\frac{1}{2}$ (0 or 1 inversion, equally likely) ✓; the maximum possible is $\binom{n}{2}$, attained by the reversed array, and the mean sits at exactly half of it, as symmetry demands (reversing a permutation maps $X \mapsto \binom{n}{2} - X$).

(b) Claim. If $a > b^k$ then $T(n) = \Theta(n^{\log_b a})$.

Take $n = b^L$, so $L = \log_b n$. Unrolling the recurrence L times:

$$T(n) = \sum_{i=0}^{L-1} a^i \, c\Big(\frac{n}{b^i}\Big)^k + a^L \, T(1) = c\, n^k \sum_{i=0}^{L-1} \Big(\frac{a}{b^k}\Big)^i + d\, a^L.$$

Note $a^L = a^{\log_b n} = n^{\log_b a}$. Write $r = a/b^k$; the hypothesis $a > b^k$ gives $r > 1$, so the geometric sum is dominated by its *largest* (last) term:

$$\sum_{i=0}^{L-1} r^i = \frac{r^L - 1}{r - 1} = \Theta(r^L), \qquad r^L = \left(\frac{a}{b^k}\right)^{\log_b n} = \frac{n^{\log_b a}}{n^k}.$$

Substituting,

$$c\,n^k \cdot \Theta(\frac{n^{\log_b a}}{n^k}) = \Theta(n^{\log_b a}),$$

and the second term $d\,n^{\log_b a}$ is of the same order. Hence

$$\boxed{T(n) = \Theta(n^{\log_b a}).}$$

(For general n, monotonicity of T — given in the question — lets you sandwich n between consecutive powers of b, and $\lceil\cdot\rceil$ effects change nothing since $b^{\log_b a}$ is a constant factor.) ■

Interpretation. $r > 1$ means the total work *grows* geometrically down the recursion tree, so the bottom level — the $a^L = n^{\log_b a}$ leaves — dominates. That one sentence is the whole theorem, and is worth writing.

Q4. [**10** marks]

Algorithm — two-pointer sweep.

1. $j \leftarrow 1$, *best* $\leftarrow 0$, *bestStart* $\leftarrow 1$.
2. For $i = 1, 2, \ldots, n$:
   (a) While $j \le n$ and $a_j \le a_i + 1$: $j \leftarrow j + 1$.
   (b) Now the interval $[a_i,\, a_i + 1]$ contains exactly $a_i, a_{i+1}, \ldots, a_{j-1}$, so its count is $j - i$. If $j - i >$ *best*, set *best* $\leftarrow j - i$ and *bestStart* $\leftarrow i$.
3. Return the interval $[a_{bestStart},\, a_{bestStart} + 1]$ and the count *best*.

Correctness

Step 1: it suffices to consider intervals whose left endpoint is a point of S. Let $I = [t, t+1]$ be any unit interval and let a_i be the *smallest* point of S inside I (if I contains no point, it is not a candidate for the maximum unless the maximum is 0). Then $a_i \ge t$, so $a_i + 1 \ge t + 1$, and every point of $S \cap I$ lies in $[a_i,\, t+1] \subseteq [a_i,\, a_i + 1]$. Hence

$$|S \cap I| \;\le\; \big|S \cap [a_i, a_i + 1]\big|,$$

so no interval beats the best one of the restricted form. Since the algorithm examines $[a_i, a_i{+}1]$ for *every* i, it examines an optimal interval.

Step 2: the count computed is correct. S is increasing, so the points in $[a_i, a_i + 1]$ are consecutive, starting at a_i. The inner loop advances j to the first index with $a_j > a_i + 1$, so the contained points are exactly $a_i, \ldots, a_{j-1}$, a count of $j - i$.

Step 3: j never needs to move backwards. $a_i + 1$ is increasing in i, so the first index exceeding it is non-decreasing in i. Hence carrying j over from the previous iteration — rather than restarting it — is valid. ■

Time complexity

i increases n times in total; j increases at most n times *in total across the whole run* (it never decreases and is capped at $n+1$). Every other operation is $O(1)$. So the total is $\boxed{O(n)}$, with $O(1)$ extra space.

Trap — do not restart the inner pointer

The naive "for each i, scan forward from i" is $O(n^2)$ in the worst case (e.g. all n points inside one unit interval). The linear bound comes *entirely* from the observation in Step 3 that j is monotone. Say it explicitly — that is where the marks are.

Q5. [**10** marks]

Assumption. The elements of S are positive (otherwise shift them; the problem's mention of the total m signals this reading). Write $S = \{s_1, \ldots, s_n\}$ and set $B = \min(x, m)$ — if $x \geq m$ the answer is all of S, so we may assume the target is capped at m.

Algorithm — subset-sum dynamic programming.

1. Boolean table $R[0..B]$, with $R[0] =$ true and $R[s] =$ false otherwise. (Also keep $take[s]$, the index of the element used to first reach s, for recovery.)
2. For each element $s_t \in S$, for $s = B$ **down to** s_t: if $R[s - s_t]$ and not $R[s]$, set $R[s] \leftarrow$ true and $take[s] \leftarrow t$.
3. Let $s^\star = \max\{s \leq B : R[s] = \text{true}\}$ — scan R downwards from B.
4. Recover the subset by repeatedly following $take$: from $s^\star$, output element $take[s^\star]$ and move to $s^\star - s_{take[s^\star]}$, until reaching 0.

Correctness

Invariant. After processing the first t elements, $R[s]$ is true exactly when some subset of $\{s_1, \ldots, s_t\}$ sums to s.
Base: with no elements only $s = 0$ is reachable. *Step:* a subset of the first t elements summing to s either omits s_t (so s was already reachable) or includes it (so $s - s_t$ was reachable using the first $t - 1$), which is precisely the update rule.
Why the loop runs downwards. Iterating s from high to low guarantees that $R[s - s_t]$ still refers to the table *before* s_t was processed. Ascending order would let s_t be used repeatedly, solving the *unbounded* knapsack instead.
Since every achievable sum $\leq B$ is marked, $s^\star$ is the largest achievable sum not exceeding x — i.e. the closest from below. ■

Time complexity

n elements, each sweeping a table of size $B + 1$:

$$\boxed{O(n\,B) = O(n \min(x, m)) \;\leq\; O(nm)}$$

time, $O(B)$ space (the *take* array costs the same). Recovery is $O(n)$.

Exam facts — say the word "pseudo-polynomial"

Subset-sum is **NP-hard**, so no algorithm polynomial in the *input length* is expected; $O(nm)$ is polynomial in the *value* m, not in its bit-length $O(\log m)$. That is what "pseudo-polynomial" means, and it is exactly why the question says "*efficient*" rather than naming a complexity. Contrast with the brute force over all 2^n subsets.

Q6. [**10** marks]

Algorithm — modified merge sort. `Count`$(A[\ell..r])$ returns the number of significant pairs inside the range *and* leaves the range sorted ascending.

1. If $\ell \geq r$, return 0.

2. $m \leftarrow \lfloor(\ell + r)/2\rfloor$; recursively $c_L \leftarrow \texttt{Count}(A[\ell..m])$ and $c_R \leftarrow \texttt{Count}(A[m{+}1..r])$. Both halves are now sorted.
3. **Cross-count.** Let $L = A[\ell..m]$ and $R = A[m{+}1..r]$. Walk a pointer q over R in increasing order and maintain a pointer p into L at the first position with $L[p] > 2R[q]$. Add $|L| - p$ to a counter c_X. Because $2R[q]$ increases with q, the pointer p only moves **forward**.
4. Merge L and R into $A[\ell..r]$ in the standard way.
5. Return $c_L + c_R + c_X$.

Correctness

Every significant pair is counted exactly once. Merge sort splits by **index**, so for a pair (i, j) with $i < j$ exactly one of three things holds: both indices lie in the left half (counted by the recursive call on L), both in the right half (counted by the call on R), or i is in the left half and j in the right half (counted in step 3). The third case is correct because *every* left-half index is smaller than *every* right-half index, so the condition $i < j$ is automatic and only $A[i] > 2A[j]$ needs testing.
The cross-count is correct. With L sorted ascending, $\{p : L[p] > 2R[q]\}$ is an upward-closed set, so its size is $|L| - p$ where p is the first such position — which is what the pointer tracks.
Sorting the halves is harmless. Whether a pair is significant depends only on the two *values* and on which half each index came from, never on the order within a half. So reordering inside a half after its own pairs have been counted changes nothing.
Monotonicity is what makes the sweep legal. R is scanned in increasing order and $z \mapsto 2z$ is increasing, so the threshold $2R[q]$ increases and p never needs to retreat. ■

Time complexity

Steps 3 and 4 are each one linear pass, so

$$T(n) = 2T(n/2) + O(n) \quad \Longrightarrow \quad \boxed{O(n \log n)}$$

by the Master Theorem, Case 2 ($a = b = 2$, $n^{\log_b a} = n$, $f(n) = \Theta(n)$).

Exam facts — one skeleton, many questions

Identical structure solves: counting inversions ($A[i] > A[j]$), this problem ($A[i] > 2A[j]$), and practice Q5 ($A[i] > 3A[j] + 5$). **The only requirement is that the threshold function $g(A[j])$ be monotone increasing** — that is what permits the single forward pointer. If it were not monotone you would need a balanced BST or BIT per merge, giving $O(n \log^2 n)$.

Q7. [**10** marks]

Algorithm — two applications of median-of-medians.

1. Find the median M of S by median-of-medians (§3). $O(n)$
2. Form the array of distances $d_i = |s_i - M|$ for every $s_i \in S$. $O(n)$
3. Find the k-th smallest value δ among $d_1, \ldots, d_n$, again by median-of-medians. $O(n)$
4. Output every s_i with $d_i < \delta$; then top the answer up to exactly k elements using elements with $d_i = \delta$. $O(n)$

Correctness

Step 3 gives the value δ such that exactly k of the d_i are $\leq \delta$ when counted with multiplicity. Step 4 outputs a set K of size exactly k consisting of elements with the k smallest distances: every element with $d_i < \delta$ must be in any optimal answer, and the remaining slots are filled with elements at distance exactly δ, which are interchangeable.
Hence for any $s \in K$ and $s' \notin K$ we have $|s - M| \leq |s' - M|$, which is precisely the statement that K consists of k numbers nearest the median. (With S a set of *distinct* numbers, ties $d_i = d_j$ can still occur — one element on each side of M — so step 4's tie-handling is genuinely needed, and any choice is optimal.) ■

Time complexity

Four phases, each $O(n)$ in the worst case:

$$\boxed{O(n)}.$$

The whole point is that *both* selections use the worst-case-linear algorithm. Sorting S ($O(n \log n)$) or sorting the d_i would miss the bound.

Trap — do not sort, and do not use randomised quickselect

Two tempting wrong answers:

- "Sort S, take the k values around the middle": $O(n \log n)$, fails the required $O(n)$.
- "Use randomised quickselect": that is $O(n)$ *expected*, but $O(n^2)$ in the worst case. The question says $O(n)$-time, so you need the deterministic median-of-medians guarantee. Name it.

Also note the answer is *not* simply "the k elements around position $n/2$ in sorted order" unless you have already sorted — and you must not.

17 Practice Set

Fresh problems, in the style and at the level of the 2022 paper — deliberately near-misses of it, so pattern-matching cannot substitute for understanding. Total available: 105 marks; treat any 60 of them as a full paper. Solutions begin in §18 — attempt first.

Q1. [**5+5** marks]

(a) A is an unsorted array of size n with $A[i] \in \{0, 1, \ldots, k-1\}$ for a fixed constant k. Give an $O(n)$-time, $O(1)$-extra-space algorithm to sort A. What changes if $k = \Theta(n)$?

(b) An array of n items is given, each coloured Red, White or Blue. Rearrange the array *in place* in $O(n)$ time and $O(1)$ extra space so that all Reds come first, then all Whites, then all Blues. State the loop invariant.

Q2. [**7+3** marks]

(a) Given n points in the plane, design an $O(n \log n)$ algorithm to compute the convex hull. Prove it correct and justify the complexity.

(b) Prove that any comparison-based algorithm computing the convex hull of n points requires $\Omega(n \log n)$ time.

Q3. [**5+5** marks]

(a) Let A be a uniformly random permutation of n distinct numbers. An *inversion* is a pair (i, j) with $i < j$ and $A[i] > A[j]$; let X be the number of inversions. Compute $\mathbb{E}(X)$. Then give an $O(n \log n)$ algorithm to count the inversions of a *given* array.

(b) Let $T(n) = a\,T(n/b) + c\,n^k$ with $T(1) = d$, $a, c > 0$, $k, d \geq 0$, $b \geq 2$. Prove that if $a < b^k$ then $T(n) = \Theta(n^k)$.

Q4. [**10** marks]

$S = \langle a_1 < a_2 < \cdots < a_n \rangle$ is an increasing sequence of points on the real line, and $k \leq n$ is a positive integer. Give an $O(n)$-time algorithm that finds the **shortest** interval containing at least k points of S. Prove correctness.

Q5. [**10** marks]

Let A be an array of n distinct reals. Call (i, j) a *heavy pair* if $i < j$ and $A[i] > 3A[j] + 5$. Design an $O(n \log n)$ algorithm to count the heavy pairs.

Q6. [**10** marks]

There are n jobs; job i occupies the interval $[s_i, f_i]$ and has weight $w_i > 0$. Find a maximum-weight set of pairwise non-overlapping jobs in $O(n \log n)$. Prove optimal substructure. What does this problem become in the language of §12, and why is that significant?

Q7. [**10** marks]

Let T be a tree on n vertices. Give an $O(n)$ algorithm that computes the eccentricity $\mathrm{ecc}(v) = \max_u d(v, u)$ for *every* vertex v simultaneously. Prove it correct.

Q8. [**5+5** marks]

(a) Prove that a simple planar graph with $n \geq 3$ vertices has at most $3n - 6$ edges, and deduce that K_5 is not planar.

(b) Prove that a *triangle-free* simple planar graph with $n \geq 3$ vertices has at most $2n - 4$ edges, and deduce that $K_{3,3}$ is not planar.

Q9. [**10** marks]

G is a chordal graph on n vertices and m edges, and a perfect elimination ordering of G is given. Give an $O(n + m)$ algorithm that computes $\chi(G)$ and $\omega(G)$, and prove that they are equal.

Q10. [**10** marks]

Given a set S of n distinct reals, decide whether there exist four *distinct* elements $x, y, z, w \in S$ with $x + y = z + w$. Give an algorithm and analyse it. (Aim for $O(n^2)$ up to the cost of a dictionary.)

Q11. [**10** marks]

You are given the **preorder** traversal of a binary search tree on n distinct keys. Reconstruct the tree in $O(n)$ time and prove correctness. Explain why the analogous problem for a general binary tree is impossible.

Q12. [**5** marks]

Prove that any comparison-based algorithm that merges two sorted arrays of size n each requires at least $2n - 1$ comparisons in the worst case.

18 Solutions to the Practice Set

Q1. [**5+5** marks]

(a) Counting sort. Keep an array $C[0..k-1]$ of counters (size k, a constant, hence $O(1)$ space). One pass over A incrementing $C[A[i]]$; then one pass writing $C[0]$ copies of 0, then $C[1]$ copies of 1, and so on. Time $O(n + k) = O(n)$; extra space $O(k) = O(1)$.

Correctness: the output is non-decreasing by construction, and it is a permutation of the input because each value v is written exactly $C[v] = \#\{i : A[i] = v\}$ times.

No contradiction with the $\Omega(n \log n)$ bound: this is not a comparison sort — it uses the values as array indices.

If $k = \Theta(n)$: the time is still $O(n + k) = O(n)$, but the counter array now needs $\Theta(n)$ space, so the $O(1)$-space claim fails. It becomes an $O(n)$-time, $O(n)$-space algorithm.

(b) Dutch national flag, three pointers. Maintain $lo = 0$, $mid = 0$, $hi = n - 1$ and loop while $mid \le hi$:

- $A[mid]$ Red: swap $A[lo] \leftrightarrow A[mid]$; lo++; mid++.
- $A[mid]$ White: mid++.
- $A[mid]$ Blue: swap $A[mid] \leftrightarrow A[hi]$; hi−− (do *not* advance mid).

Loop invariant: $A[0, lo)$ all Red, $A[lo, mid)$ all White, $A(hi, n)$ all Blue, and $A[mid, hi]$ unexamined.

Correctness: each branch restores the invariant — the Blue case cannot advance mid because the element swapped in from hi is still unexamined. On termination $mid > hi$, so the unexamined region is empty and the three blocks tile the array.

Complexity: every iteration either increments mid or decrements hi, so the quantity $hi - mid$ strictly decreases: at most n iterations, each $O(1)$. Time $O(n)$, extra space $O(1)$ (three indices). The same three-way partition is exactly what a robust quicksort/quickselect uses when duplicates are present.

Q2. [**7+3** marks]

(a) Andrew's monotone chain.

1. Sort the points lexicographically by (x, y): $O(n \log n)$.
2. **Lower hull:** scan left to right maintaining a stack; before pushing p, while the top two stack points q, r and p do *not* make a counter-clockwise turn (i.e. the cross product $(r-q) \times (p-q) \le 0$), pop.
3. **Upper hull:** the same scan on the reversed order.
4. Concatenate, dropping the duplicated endpoints.

Correctness: a point is a vertex of the lower hull iff it makes a strict left turn with its hull-neighbours. The stack maintains the invariant "the current stack is the lower hull of the points scanned so far": popping removes exactly the points that the new point makes non-convex, and no popped point can return to the hull, since it lies below the segment joining two points that remain. Doing this in both directions gives the full hull.

Complexity: $O(n \log n)$ for the sort; the scan pushes each point once and pops it at most once, so it is $O(n)$ amortised. Total $O(n \log n)$, which by (b) is optimal.

(b) Reduction from sorting. Given reals $x_1, \ldots, x_n$, build the points (x_i, x_i^2) in $O(n)$ time. All of them lie on the parabola $y = x^2$, which is strictly convex, so *every* input point is a hull vertex. A hull algorithm outputs the vertices in cyclic order; starting at the leftmost and reading along the lower chain gives $x_1, \ldots, x_n$ in increasing order — an $O(n)$ extraction.

So a hull algorithm running in $T(n)$ would sort in $T(n) + O(n)$. Since comparison sorting is $\Omega(n \log n)$ (§6), $T(n) + O(n) = \Omega(n \log n)$, hence $T(n) = \Omega(n \log n)$. ∎

Q3. [**5+5** marks]

(a) Expectation. For each pair $i < j$ let $X_{ij} = \mathbf{1}\{A[i] > A[j]\}$, so $X = \sum_{i<j} X_{ij}$. In a uniformly random permutation the two orderings of any fixed pair are equally likely, so $\mathbb{E}(X_{ij}) = \frac{1}{2}$. By linearity (no independence needed):

$$\mathbb{E}(X) = \binom{n}{2} \cdot \frac{1}{2} = \boxed{\frac{n(n-1)}{4}}.$$

Counting algorithm — modified merge sort. Sort recursively, and during the merge of the sorted halves L and R: whenever an element of R is output while t elements of L remain unconsumed, add t to the counter (that element of R is inverted with exactly those t elements).

Correctness: every pair (i, j) with $i < j$ has both indices in L, both in R, or split. The first two are counted by the recursive calls, the third exactly once during this merge, and L holds precisely the smaller indices. So each inverted pair is counted once.

Complexity: $T(n) = 2T(n/2) + O(n) = O(n \log n)$ (Master Theorem, Case 2).

(b) Unroll for $n = b^L$, $L = \log_b n$, exactly as in §2:

$$T(n) = c\,n^k \sum_{i=0}^{L-1} \left(\frac{a}{b^k}\right)^i + d\,n^{\log_b a}.$$

Put $r = a/b^k$. Now $a < b^k$ gives $r < 1$, so the geometric series *converges*:

$$\sum_{i=0}^{L-1} r^i \;\le\; \sum_{i=0}^{\infty} r^i = \frac{1}{1-r} = \Theta(1),$$

and it is at least 1. Hence the first term is $\Theta(n^k)$. For the second, $a < b^k$ gives $\log_b a < k$, so $n^{\log_b a} = o(n^k)$. Therefore

$$T(n) = \Theta(n^k) + o(n^k) = \boxed{\Theta(n^k)}.$$

(The lower bound is immediate anyway: the root alone contributes $c\,n^k$.) ■*Interpretation:* $r < 1$ means the work *shrinks* geometrically down the tree, so the root dominates — Master Theorem Case 3.

Q4. [**10** marks]

Algorithm. For $i = 1, \ldots, n - k + 1$ compute $a_{i+k-1} - a_i$ and return the minimum (and the interval attaining it). $O(n)$ time, $O(1)$ space.

Correctness. Let I be any interval containing at least k points of S. Since S is sorted, the points of S inside I are **consecutive**: $a_i, a_{i+1}, \ldots, a_j$ with $j - i + 1 \ge k$. Then

$$|I| \;\ge\; a_j - a_i \;\ge\; a_{i+k-1} - a_i,$$

the second inequality because $j \ge i + k - 1$ and the a's increase. So no interval beats the best window. Conversely each window $[a_i, a_{i+k-1}]$ genuinely contains the k points $a_i, \ldots, a_{i+k-1}$, so the minimum over windows is attained. ■

Remark. The 2022 paper asked the dual — *fixed* length 1, maximise the count. That one needs a two-pointer sweep: advance j while $a_j \le a_i + 1$, take $j - i$, then advance i. Both pointers only move forward, so it is $O(n)$. Know both directions.

Q5. [**10** marks]

Algorithm — merge-sort variant. `CountAndSort`$(A[\ell..r])$:

1. If $\ell \ge r$, return 0.
2. Split at the midpoint; recursively count-and-sort each half, obtaining sorted L, R and their counts.
3. **Cross count.** Both halves are now sorted ascending. Walk j over R in increasing order; maintain a pointer p into L marking the first position with $L[p] > 3R[j] + 5$. Add $|L| - p$ to the count. Since $3R[j] + 5$ increases with j, p is non-decreasing — one monotone sweep.
4. Merge L and R normally and return the total count.

Correctness. Every pair (i, j) with $i < j$ lies wholly in the left half, wholly in the right half, or is split with i in the left and j in the right — because merge sort splits by *index*, so all left-half indices precede all right-half indices. The first two cases are handled by recursion; the third is counted in

step 3, which for each j counts exactly the L-elements exceeding $3A[j] + 5$. Sorting the halves is harmless: the count for a split pair depends only on the two *values*, not on their order within a half.

Complexity. Steps 3 and 4 are each a single linear sweep, so $T(n) = 2T(n/2)+O(n) = \boxed{O(n \log n)}$ by the Master Theorem, Case 2.

Remark. The same skeleton solves 2022 Q6 ($A[i] > 2A[j]$) and inversion counting ($A[i] > A[j]$): only the threshold function changes. It must be **monotone** in $A[j]$ for the single-pointer sweep to be valid.

Q6. [**10** marks]

Weighted interval scheduling.

1. Sort jobs by finishing time so $f_1 \le f_2 \le \cdots \le f_n$: $O(n \log n)$.
2. For each j compute $p(j) = \max\{i < j : f_i \le s_j\}$ (or 0), by binary search in the sorted finish times: $O(\log n)$ each, $O(n \log n)$ total.
3. DP with $\mathrm{opt}(0) = 0$ and
$$\mathrm{opt}(j) = \max\Big\{ w_j + \mathrm{opt}(p(j)),\ \ \mathrm{opt}(j-1) \Big\}.$$
4. Answer $\mathrm{opt}(n)$; recover the set by tracing which branch achieved each maximum.

Optimal substructure. Consider an optimal solution O_j for jobs $1, \ldots, j$.

- If job $j \in O_j$: no job i with $p(j) < i < j$ can be in O_j, since every such job finishes after s_j and starts before f_j, hence overlaps j. So $O_j \setminus \{j\}$ is a feasible solution on $1, \ldots, p(j)$, and it must be optimal there — otherwise replacing it by a better one and re-adding j would beat O_j. Value $w_j + \mathrm{opt}(p(j))$.
- If job $j \notin O_j$: O_j is a feasible solution on $1, \ldots, j-1$ and must be optimal there by the same exchange argument. Value $\mathrm{opt}(j-1)$.

These two cases are exhaustive, so the maximum of the two values is $\mathrm{opt}(j)$; induction on j completes the proof. ■

Complexity. $O(n \log n)$ (sort + binary searches) + $O(n)$ (DP) = $\boxed{O(n \log n)}$.

The connection. Build the interval graph G whose vertices are the jobs, with an edge between overlapping jobs. A set of pairwise non-overlapping jobs is exactly an **independent set** of G, so this is **maximum-weight independent set on an interval graph**. That is significant because MWIS is NP-hard on general graphs, yet interval graphs are chordal, hence perfect, and the structure (here: a linear order by finishing time) makes the problem solvable in $O(n \log n)$ — the same phenomenon as §13, where a PEO makes maximum independent set easy on chordal graphs.

Q7. [**10** marks]

Algorithm. Three BFS runs.

1. BFS from an arbitrary s; let u be a farthest vertex.
2. BFS from u, recording $d_u(\cdot)$; let v be a farthest vertex. Then (u, v) is a diametral pair.
3. BFS from v, recording $d_v(\cdot)$.
4. Output $\mathrm{ecc}(x) = \max\{d_u(x), d_v(x)\}$ for every x.

Correctness. Step 2 gives a diametral pair by the two-BFS trick (§9). It remains to show: for *every* x and any diametral pair (u, v), $\mathrm{ecc}(x) = \max\{d(x,u), d(x,v)\}$.

"$\ge$" is trivial. For "$\le$", let P be the (unique) u–v path, let w be any vertex, and let m, m_w be the vertices of P closest to x and to w respectively. First, since $d(u,w) = d(u,m_w) + d(m_w,w) \le \mathrm{diam} = d(u,m_w) + d(m_w,v)$, we get $d(m_w,w) \le d(m_w,v)$, and symmetrically $d(m_w,w) \le d(m_w,u)$.

- If $m = m_w$: $d(x,w) = d(x,m)+d(m,w) \le d(x,m)+\max\{d(m,u),d(m,v)\} = \max\{d(x,u),d(x,v)\}$.

- If $m \neq m_w$, say m lies between u and m_w on P. Then

$$d(x,v) = d(x,m)+d(m,v) = d(x,m)+d(m,m_w)+d(m_w,v) \;\geq\; d(x,m)+d(m,m_w)+d(m_w,w) = d(x,w).$$

(The symmetric case gives $d(x,u) \geq d(x,w)$.)

Either way $d(x,w) \leq \max\{d(x,u), d(x,v)\}$ for every w, proving the claim. ■

Complexity. Three BFS runs on a tree: $\boxed{O(n)}$ (since $|E| = n-1$).

Q8. [**5+5** marks]

(a) Every face of a plane drawing of a simple graph with $n \geq 3$ is bounded by at least 3 edges, and every edge borders exactly 2 faces. Counting edge–face incidences two ways, $2E \geq 3F$, i.e. $F \leq \frac{2}{3}E$. Substituting into Euler's formula $n - E + F = 2$:

$$2 = n - E + F \;\leq\; n - E + \tfrac{2}{3}E = n - \tfrac{1}{3}E \quad\Longrightarrow\quad E \leq 3n - 6.$$

K_5**:** $n = 5$, $E = \binom{5}{2} = 10$, but $3n - 6 = 9 < 10$. So K_5 violates the bound and is not planar. ■

(b) If G is triangle-free then every face is bounded by at least 4 edges, so $2E \geq 4F$, i.e. $F \leq \frac{1}{2}E$. Euler again:

$$2 = n - E + F \leq n - E + \tfrac{1}{2}E = n - \tfrac{1}{2}E \quad\Longrightarrow\quad E \leq 2n - 4.$$

$K_{3,3}$**:** it is bipartite, hence triangle-free, with $n = 6$ and $E = 9$; but $2n - 4 = 8 < 9$. So $K_{3,3}$ is not planar. ■

Note: the first bound alone does *not* rule out $K_{3,3}$ — $3n - 6 = 12 \geq 9$. That is exactly why the triangle-free refinement is needed, and it is a favourite exam trap.

Q9. [**10** marks]

Algorithm. Let $v_1, \ldots, v_n$ be the given PEO. Process it in order; for each i compute

$$d_i = \big|N(v_i) \cap \{v_1, \ldots, v_{i-1}\}\big|$$

and give v_i the lowest-indexed colour not used by those earlier neighbours. Output $\max_i(d_i + 1)$ as both $\chi(G)$ and $\omega(G)$, together with the colouring.

Complexity. Computing all the d_i is one scan of the adjacency lists with a position array, $O(n+m)$. Choosing v_i's colour needs a scratch array of size $d_i + 1$ marking the colours of its earlier neighbours, so the total colouring work is $\sum_i O(d_i + 1) = O(n+m)$. Total $\boxed{O(n+m)}$.

Proof that $\chi = \omega$.

- **Upper bound on** χ**.** When v_i is coloured, its earlier neighbours form a *clique* (PEO property) of size d_i, so they carry d_i *distinct* colours, and some colour in $\{1, \ldots, d_i + 1\}$ is free. Hence the greedy uses at most $1 + \max_i d_i$ colours, so $\chi(G) \leq 1 + \max_i d_i$.
- **Lower bound on** ω**.** For the index i attaining the maximum, the set $\{v_i\} \cup \big(N(v_i) \cap \{v_1, \ldots, v_{i-1}\}\big)$ is a clique of size $d_i + 1$ (the earlier neighbours are mutually adjacent, and v_i is adjacent to all of them). So $\omega(G) \geq 1 + \max_i d_i$.
- **Universal inequality.** $\chi(G) \geq \omega(G)$ always.

Chaining: $\omega \leq \chi \leq 1 + \max_i d_i \leq \omega$, so all are equal:

$$\boxed{\chi(G) = \omega(G) = 1 + \max_i d_i.}$$

■ *This is the constructive proof that chordal graphs are perfect* (the argument applies verbatim to every induced subgraph, since an induced subgraph of a chordal graph is chordal and inherits a PEO).

Q10. [**10** marks]

Key observation. If $\{x, y\}$ and $\{z, w\}$ are *distinct* pairs with $x + y = z + w$, they are automatically **disjoint**: if they shared an element, cancelling it would force the other two to be equal,

impossible among distinct elements. So we need only find two distinct pairs with equal sums — distinctness of all four is then free.

Algorithm. Form all $\binom{n}{2}$ pairwise sums, each tagged with its pair. Insert them into a dictionary keyed by the sum. If any key is hit twice, output the two pairs; if no collision occurs, answer "no".

Correctness. A "yes" instance is precisely a repeated sum, by the key observation. The algorithm reports exactly that.

Complexity. $\binom{n}{2} = \Theta(n^2)$ sums to generate.

- With a hash table: $O(n^2)$ expected time, $O(n^2)$ space.
- Deterministically, by sorting the sums and scanning for adjacent duplicates: $O(n^2 \log n)$ time (which is $O(n^2 \log n) = O(n^2 \cdot \log n)$; note $\log \binom{n}{2} = \Theta(\log n)$), $O(n^2)$ space.

Remark. One can stop as soon as a collision appears, so on "yes" instances the algorithm often terminates far earlier — but the worst case (a Sidon set, where all pairwise sums are distinct) forces all $\Theta(n^2)$ sums to be examined.

Q11. [**10** marks]

Why preorder alone suffices for a BST. The inorder traversal of a BST with distinct keys is forced — it is the keys in increasing order. So preorder + inorder is available, and by the standard reconstruction (§4) that determines the tree uniquely. Sorting would cost $O(n \log n)$; the bounds method below achieves $O(n)$.

Algorithm (bounds method). Keep a global index $idx = 0$ into the preorder array P. Define

$$\texttt{Build}(lo, hi) :$$

1. If $idx = n$, or $P[idx] \notin (lo, hi)$, return `null`.
2. Let $\kappa = P[idx]$; create a node with key κ; idx++.
3. Left child $\leftarrow$ $\texttt{Build}(lo, \kappa)$; right child $\leftarrow$ $\texttt{Build}(\kappa, hi)$.
4. Return the node.

Call $\texttt{Build}(-\infty, +\infty)$.

Correctness. By the BST property, in the preorder sequence the root κ is followed first by the entire preorder of its left subtree — all of whose keys are $< \kappa$ — and then by the preorder of its right subtree, all of whose keys are $> \kappa$. So the split point is exactly the first position after κ carrying a key $> \kappa$, which is precisely what the bound test $P[idx] \in (lo, hi)$ detects. Induction on subtree size gives correctness.

Complexity. idx advances exactly once per node, and each node generates $O(1)$ calls, so the total number of calls (including those returning `null`) is $O(n)$, each doing $O(1)$ work: $\boxed{O(n)}$.

Why a general binary tree is impossible. With n distinct keys there are far more binary trees than preorder sequences can distinguish: concretely, a preorder "a, then b" is consistent with b being either the left child or the right child of a, two genuinely different trees producing identical preorder output. Hence no algorithm can reconstruct. The BST case escapes only because the extra ordering constraint supplies the missing inorder information.

Q12. [**5** marks]

Adversary argument. Consider inputs in which the correct merged output alternates between the two arrays:

$$a_1 < b_1 < a_2 < b_2 < \cdots < a_n < b_n.$$

The merged sequence has $2n$ elements, hence $2n - 1$ adjacent pairs, and each adjacent pair consists of one element from each array (by the alternation).

Suppose some algorithm halts having never compared some adjacent pair (u, v) from different arrays, where u immediately precedes v in the true output. Build a second input identical except that the values of u and v are swapped. This second input is still a legal pair of sorted arrays (swapping two adjacent values across the two arrays preserves the sorted order within each array),

and it is consistent with *every* comparison the algorithm actually made — the only comparison whose outcome would change is the one between u and v, which was never performed. Yet the correct output differs, since u and v are transposed. So the algorithm gives a wrong answer on one of the two inputs.

Hence a correct algorithm must compare all $2n-1$ adjacent pairs, requiring at least $2n-1$ comparisons in the worst case. ■

Matching upper bound: standard merging uses at most $2n-1$ comparisons, so the bound is tight.

Sections 1–13 follow the lecture notes exactly; the practice set is modelled on the mid-semester examination of 22 September 2022.